

Relatório 6: Uso de escalonador de tarefas

João Bruno dos Santos, João Lucas Medina Kormann

Universidade Federal de Santa Catarina

I. INTRODUÇÃO

Este relatório tem o objetivo de apresentar a metodologia aplicada para a resolução dos problemas do Laboratório 6 da disciplina Organização de Computadores I, ministrada pelo professor Marcelo Daniel Berejuck, utilizando a linguagem Assembly para a arquitetura MIPS (Microprocessor without Interlocked Pipelined Stages), o simulador MARS (MIPS Assembler and Runtime Simulator) e também o kernel desenvolvido pelo aluno Pedro Henrique Tesman. O trabalho consiste no desenvolvimento de um programa concorrente que, utilizando o digital lab sim, apresenta um contador de 1 em 1 segundo no display à esquerda e um contador de 3 em 3 segundos no display à direita.

II. IMPLEMENTAÇÃO DO MÉTODO DE NEWTON

A. PASSOS INICIAIS

De forma geral, a implementação para este projeto foi bem simples. Inicialmente começamos entendendo melhor o funcionamento do kernel, obtendo mais informações através dos dois arquivos disponibilizados no ambiente virtual moodle. Onde, de forma resumida, sua implementação parte da alteração do número máximo de threads juntamente com a criação de um arquivo .asm para cada thread a ser executada.

```
# CONFIGURAÇÃO:
#
# Essas duas constantes simbólicas abaixo definem a lista de threads que o kern
# vai estar preparado para gerenciar.
# A primeira, max_th, diz respeito ao número de threads na lista estática, enqu
# a segunda, thread_v, dita os labels de cada thread. Podem haver labels repeti
# (duas threads com mesmo código), dependendo da aplicação pode ser algo útil..

.equ max_th 2
.equ thread_v led_1, led_2
```

Figura 1 - Máximo de threads e seus nomes

B. LÓGICA CONTADORES

Partindo para o arquivo das threads em si, ambos ficaram muito semelhantes, sendo sua única diferença o valor referente à quantidade de tempo de cada contador.

Para a lógica do contador, utilizamos uma proposta muito parecida com a utilizada pelo próprio arquivo do kernel, onde, através do comando 30 do mars, obtemos o tempo de execução do processo até aquele determinado ponto, somando então este valor ao desvio (*offset*) demandando (1 ou 3 segundos) e então executando o comando 30 em *loop* até que seja alcançado esse valor da soma, aumentando assim o contador em 1 e recomeçando o processo.

C. PASSO A PASSO

```
.include "labels.asm"

.globl led_1

.data
led_end: .word 0xFFFF0011

tabela_segmentos:
    .byte 0x3F,0x06,0x5B,0x4F
    .byte 0x66,0x6D,0x7D,0x07
    .byte 0x7F,0x6F,0x77,0x7C
    .byte 0x39,0x5E,0x79,0x71
```

Figura 2 - Inicialização Led

Como primeiro passo, utilizamos o `.include` e `.globl` necessários para o funcionamento do kernel e criamos o `led_end` e `tabela_segmentos` para salvar o valor do endereço do led à esquerda e os valores referentes à representação dos números de 0 a 15 no display de 7 segmentos respectivamente.

```

.text
led_1:
    la    $t7, tabela_segmentos

    la    $t1, led_end
    lw    $t1, 0($t1)          # agora $t1 = 0xFFFF0010

loop_led:
    lb    $t6, 0($t7)
    sb    $t6, 0($t1)          # escreve byte no LED
    addi  $t7, $t7, 1

    li    $a0, 1000
    jal   DELAY_PROC

    beq   $t6, 0x71, led_1
    j     loop_led

```

Figura 3 - Inicio Led

Começando a lógica em si, carregamos o endereço da tabela de segmentos em \$t7 ao passo que carregamos também o valor referente ao endereço do led em \$t1. Para o *loop_led*, iniciamos carregando o valor referente ao número atual traduzido para o display de 7 segmentos no registrador \$t6 e então utilizamos este mesmo registrador para salvar este valor no endereço do valor de \$t1, exibindo-o assim no led à esquerda, após isso somamos mais 1 ao endereço \$t7 para assim caminharmos para o próximo byte referente ao próximo valor. Por fim, carregamos 1000 em \$a0 para simular 1 segundo, utilizamos um jal para *delay_proc* que será explicado adiante e, na volta, comparamos \$t6 com 0x71, valor referente a F (15), caso verdadeiro, o programa volta para o começo reiniciando assim todo o processo do contador. Caso falso, o programa retorna para *loop_led* e executa novamente este processo.

```

DELAY_PROC:
    move  $t0, $a0
    li    $v0, 30
    syscall
    add   $t0, $t0, $a0

loop:
    li    $v0, 30
    syscall
    blt   $a0, $t0, loop

    jr    $ra

```

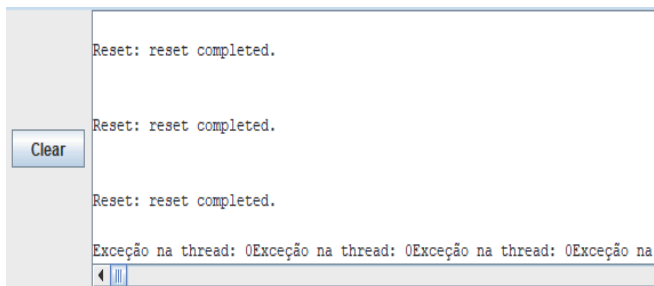
Figura 4 - Delay_proc e loop

Por fim, no *delay_proc* salvamos o valor de desvio temporariamente em \$t0, obtemos o tempo total de execução até o momento através de 30 carregado em \$v0 e então adicionamos este tempo (carregado em \$a0) com o desvio (carregado em \$t0). Partindo para o *loop* então, executamos várias vezes o comando 30 enquanto o valor de \$a0, que é atualizado sempre que o comando 30 é chamado, não ultrapasse o valor inicial + desvio. Quando este é ultrapassado, retornamos para o jal e executamos as duas verificações citadas anteriormente.

III. RESULTADOS E LIMITAÇÕES

Em geral, foram conquistados resultados bastante satisfatórios, tendo obtido êxito na proposta demandada pelo laboratório de forma simples e intuitiva. Como dificuldade pontual apenas existiu o fato de algumas pequenas coisas não poderem ser tratadas da mesma forma como convencional, como exemplo, no caso onde executamos “**LW \$t1, 0(\$t1)**”, não podemos fazer isso direto como normalmente, sendo então necessário executar “**LA \$t1, led_end**” caso contrário, o programa lançada infinitas exceções de thread 0 e acabava travando o mars. Por não ser uma situação usual, demandou-se um pouco de tempo e esforço para encontrar qual era a interação que causava este erro pois não conseguimos fazer teste parte a parte já que o programa travava, porém, assim que encontrada foi fácil resolvê-la.

*A título de melhor definição e clareza para o erro que travava o sistema, este ocorria apenas com algumas configurações específicas, sendo elas o número de threads máximas 1 juntamente com o uso apenas do LW como narrado no item 3, caso fosse aumentado o número de threads, apenas aquele com o uso inadequada de LW travava ou, caso os dois possuissem, ambos travavam lançando infinitas exceções porém de forma mais controlada, não travando assim o MARS.



CÓDIGO QUE OCASIONA NO TRAVAMENTO

```
(Com
.eqv max_th 1
.eqv thread_v led_1
)
```

```
.include "labels.asm"
```

```
.globl led_1
```

```
.data
```

```
led_end: .word 0xFFFF0010
```

```
tabela_segmentos: .byte
0x3F,0x06,0x5B,0x4F
```

```
.byte
0x66,0x6D,0x7D,0x07
```

```
.byte
0x7F,0x6F,0x77,0x7C
```

```
.byte
0x39,0x5E,0x79,0x71
```

```
.text
```

```
la $t7, tabela_segmentos
```

```
led_1:
```

```
lw $t1 led_end
```

```
li $a0 1000
```

```
jal DELAY_PROC
```

```
lb $t9, 0($t7)
```

```
sb $t9, 0($t1) # Salva esse byte no
```

```
endereço referenciado por t1
```

```
addi $t7, $t7, 1
```

```
##j led_1
```

```
DELAY_PROC:
```

```
move $t0 $a0
```

```
li $v0 30
```

```
syscall
```

```
add $t0 $t0 $a0
```

```
loop:
```

```
li $v0 30
```

```
syscall
```

```
blt $a0 $t0 loop
```

```
move $a0 $t0
```

```
jr $ra
```

```
#####
```